

TRACE

Final Technical Blueprint v1.0

Evidence-grounded Temporal Reconstruction and Causal Investigation Engine

Engineering Reference Document

A concise, implementation-oriented blueprint for the TRACE railway accident investigation platform. The document deliberately separates MVP/Core, Advanced, and Future/Research scope and prioritizes free, lightweight, and low-complexity technologies.

Primary design principle

No TRACE finding should exist without an evidence path: Finding → Hypothesis → Causal relationship → Events → Evidence records → Original source.

Version 1.0 • Architecture freeze candidate • 2026

Contents

Section	Topic
1	Project definition and design principles
2	Scope: MVP, Advanced, Future
3	End-to-end workflow
4	System architecture
5	Evidence and provenance layer
6	Normalization and event model
7	Temporal reconstruction engine
8	MCP evidence access layer
9	Agentic investigation engine
10	Evidence arbitration and hypothesis ranking
11	Causal graph and probabilistic reasoning
12	Counterfactual analysis
13	Evidence conflicts, gaps and anomaly rules
14	Investigator UI and reporting
15	Data/storage/API architecture
16	Technology stack and alternatives
17	Phase-wise implementation plan
18	Testing, evaluation, scalability and risks
19	Final feature freeze / build checklist

Source/design note. This blueprint incorporates the useful, low-complexity patterns identified during review of the previous ADCSIS project: case-centric workflow, evidence explorer/details, provenance, graph-path exploration, hypothesis comparison, evidence gaps, layered explanations, anomaly rules, auditability and human review. The TRACE-specific distinction is temporal reconstruction plus causal accident investigation.

1. Project Definition & Design Principles

TRACE is an AI-assisted railway accident investigation platform that integrates heterogeneous railway evidence, reconstructs a consistent accident timeline, investigates competing hypotheses, identifies causal relationships, and produces evidence-traceable findings.

Core investigation questions

- What happened? — reconstruct a unified temporal sequence.
- What evidence supports it? — correlate observations and preserve provenance.
- What could have caused it? — investigate competing hypotheses independently.
- How did factors interact? — represent and analyze causal relationships.
- What remains unknown? — expose missing and contradictory evidence.
- Why does TRACE believe this? — provide an evidence-linked explanation.

Design principles

- Evidence first: deterministic evidence processing precedes LLM reasoning.
- Evidence $\neq$ event: original artifacts, parsed records and normalized events are separate concepts.
- LLM $\neq$ evidence: model-generated reasoning never becomes evidence by itself.
- LLM confidence $\neq$ Bayesian probability: they remain separate quantities unless explicitly calibrated.
- Human-in-the-loop: TRACE assists investigation; it does not declare legal responsibility or prove causation.
- Explainability by construction: every finding stores supporting, contradicting and missing evidence references.
- Simple first: use rules and deterministic/statistical methods before expensive models.
- Modular scalability: start as a modular monolith and separate services only when workload requires it.

2. Scope: MVP vs Advanced vs Future

The architecture is intentionally broader than the first implementation. This prevents premature redesign while keeping the build realistic.

MVP / Core

Feature	Status	Preferred free/simple choice	Alternative
Case management	MVP	FastAPI + PostgreSQL	SQLite for single-user prototype
Evidence ingestion	MVP	Python + pandas + stdlib	Polars
Integrity / custody	MVP	hashlib SHA-256 + PostgreSQL audit table	SQLAlchemy event/audit layer
Evidence normalization	MVP	Pydantic	dataclasses
Event extraction	MVP	Python rules + lightweight NLP	spaCy
Temporal normalization	MVP	python-dateutil + zoneinfo	pendulum
Clock correction	MVP	NumPy/SciPy linear regression	statsmodels
Timeline	MVP	React + lightweight charting	Plotly
Rule anomalies	MVP	Python rule engine	durable_rules
Conflict detection	MVP	deterministic comparisons + optional LLM	spaCy / embeddings
Agent orchestration	MVP	LangGraph	simple async Python orchestrator
Evidence access	MVP	FastAPI + MCP server	REST-only internal adapter
Causal graph	MVP	Neo4j Community	NetworkX for smaller prototype
Reports	MVP	ReportLab	WeasyPrint

Advanced

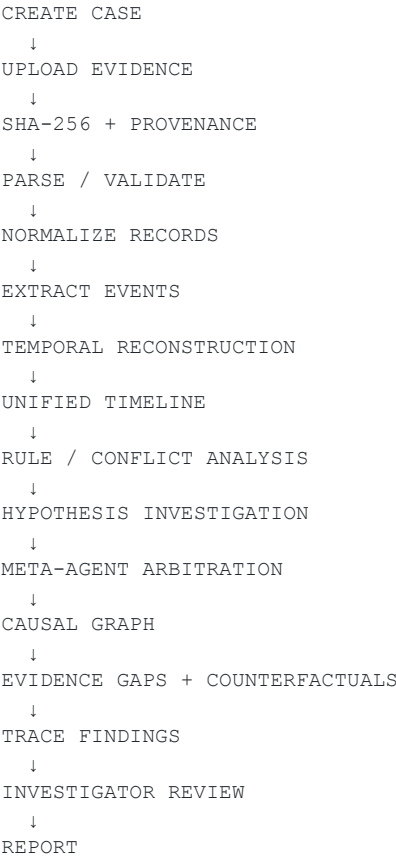
- DTW alignment for continuous signals.
- Dempster-Shafer evidence arbitration.
- Bayesian Network inference using pgmpy.
- Automatic hypothesis generation.
- Semantic evidence retrieval using sentence-transformers.
- Advanced contradiction detection.
- Model-based counterfactual analysis with DoWhy.
- Similar-case retrieval and investigator feedback.
- Dynamic agent spawning.

Future / Research

- Automated causal discovery (PC/GES or related methods).
- Real ETCS/ATC/SCADA integration and live streams.
- CCTV/audio evidence.
- Digital twin and physical simulation.
- Cross-case causal pattern discovery.
- Distributed high-volume deployment and real-time investigation.

3. End-to-End Investigation Workflow

Canonical workflow



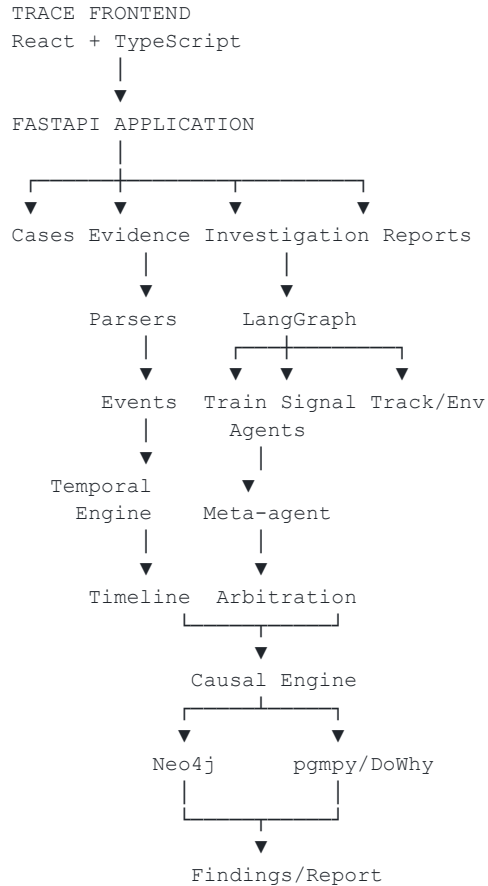
Investigation state

Each stage should persist structured state so a failed or expensive stage can be rerun without reprocessing the entire case.

State	Produced by	Consumed by
Case	Case manager	All modules
EvidenceArtifact	Ingestion	Provenance / retrieval
EvidenceRecord	Parser	Normalizer
Event	Normalizer	Temporal / agents / graph
Timeline	Temporal engine	Agents / UI / report
HypothesisFinding	Domain agents	Meta-agent
Finding	Meta/causal engine	UI / report
AuditEvent	All state-changing modules	Audit UI

4. System Architecture

TRACE should begin as a modular monolith. Components are separated by package/module boundaries and API contracts, not by deployment complexity.



Storage separation

- PostgreSQL: case metadata, evidence metadata, records/events, hypotheses, findings, audit logs.
- Object/file storage: original evidence and generated reports.
- Neo4j: event/factor/outcome relationships and causal paths.
- Vector store (advanced): semantic retrieval only if needed.

Scalability rule: Do not introduce microservices, Kubernetes or distributed infrastructure during MVP. If processing becomes asynchronous, add a lightweight job queue/worker layer first; split services only after real bottlenecks appear.

5. Evidence & Provenance Layer

Evidence is preserved as an immutable source artifact while downstream modules operate only on processed representations.

Evidence hierarchy

```

EvidenceArtifact
  ↓
EvidenceRecord
  ↓
Normalized Event

```

EvidenceArtifact fields

- evidence_id
- case_id
- filename
- source_type
- file_size
- sha256
- acquisition_time
- source_metadata
- processing_status
- parser_version
- custody_history

Core tools

Feature	Status	Preferred	Alternative
File hashing	MVP	Python hashlib	OpenSSL CLI
Metadata	MVP	Pydantic + Python	SQLAlchemy
Audit trail	MVP	PostgreSQL table	SQLite table
Original files	MVP	Local filesystem	MinIO / S3
PDF/text parsing	MVP	PyMuPDF / pypdf	pdfplumber
CSV/JSON parsing	MVP	pandas + json	Polars

Integrity workflow

```

Upload
  → calculate SHA-256
  → store hash + metadata
  → parse into derived records
  → retain original
  → verify hash when evidence is re-accessed

```

The previous project used SHA-256 and chain-of-custody logging as a core forensic capability; TRACE retains that pattern but applies it to railway evidence.

6. Normalization & Event Model

The event model is the central contract between heterogeneous railway data and every downstream component.

Example normalized event

```
{
  "event_id": "EVT-001245",
  "event_type": "SIGNAL_STATE_CHANGE",
  "timestamp": "...",
  "source_id": "SIGNAL-01",
  "entity_id": "TRAIN-102",
  "location": {"track": "T12", "position": 1832},
  "attributes": {"signal": "S42", "state": "RED"},
  "temporal_confidence": 0.96,
  "evidence_refs": ["EVID-004"]
}
```

Normalization pipeline

```
Raw source
→ schema detection
→ field mapping
→ timestamp parsing
→ entity normalization
→ unit normalization
→ event extraction
→ evidence references
```

Tools

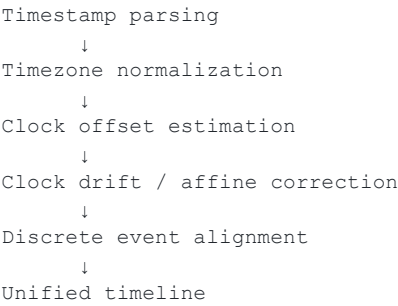
Feature	Status	Preferred	Alternative
Schema validation	MVP	Pydantic	JSON Schema
Tabular normalization	MVP	pandas	Polars
Light NLP	MVP	regex + rule patterns	spaCy
Entity resolution	MVP	normalized IDs + lookup tables	RapidFuzz
Units	MVP	explicit conversion functions	pint

Important rule: preserve both the original value and the normalized value when a transformation occurs. This allows investigators to inspect how TRACE changed a source record.

7. Temporal Reconstruction Engine

Temporal reconstruction is TRACE's main technical differentiator. The engine converts heterogeneous timestamps and asynchronous observations into a unified investigation timeline.

MVP pipeline



Continuous signals

DTW should not be applied to every event. Discrete events such as signal changes, brake actions and maintenance records should use timestamp/anchor matching. DTW is reserved for asynchronous continuous signals such as speed, acceleration or braking curves.

Feature	Status	Preferred	Alternative
Time parsing	MVP	python-dateutil + zoneinfo	Pendulum
Offset estimation	MVP	NumPy/SciPy	statsmodels
Anchor matching	MVP	Python + nearest-time logic	SciPy KD-tree
Clock drift	MVP	scipy.stats.linregress	statsmodels OLS
DTW	Advanced	tslearn	dtaidistance
Timeline display	MVP	Plotly React	D3.js

Correction model

```
corrected_time = a × raw_time + b

a = drift factor
b = clock offset
```

Every corrected event stores original timestamp, corrected timestamp, estimated offset/drift and temporal confidence.

8. MCP Evidence Access Layer

MCP is an evidence/context access interface for reasoning components—not the computational center of TRACE.

Recommended tools

Tool	Purpose	MVP?
query_evidence	Retrieve evidence records using structured filters	YES
get_event	Retrieve one event and provenance	YES
get_events	Retrieve related events	YES
get_timeline	Retrieve relevant timeline window	YES
get_source_metadata	Retrieve source information and integrity status	YES
get_evidence_provenance	Show source-to-record lineage	YES
get_evidence_gaps	Return known missing evidence	YES
get_related_events	Expand an event context	Advanced

Technology

Feature	Status	Preferred	Alternative
MCP server	MVP	Python MCP SDK	FastMCP if preferred
Agent client	MVP	LangGraph tool integration	Direct Python adapter
Underlying API	MVP	FastAPI	REST-only
Authentication	Advanced	API key / JWT	OAuth2

Security rule: agents should primarily have read access to evidence. Any mutation should be performed through controlled backend services rather than arbitrary agent tool calls.

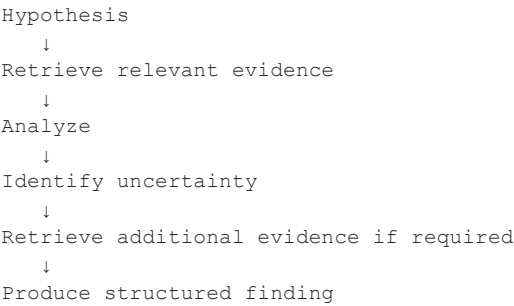
9. Agentic Investigation Engine

LangGraph coordinates independent domain investigations over shared structured state.

Recommended domain agents

Agent	Primary questions
Train / Driver Agent	Speed, braking, driver actions, operational response, train-side anomalies
Signalling / Control Agent	Signal state, interlocking, ATC/ETCS behaviour, warnings
Track / Infrastructure Agent	Track condition, maintenance history, equipment/infrastructure anomalies
Environment Agent	Rainfall, visibility, temperature, weather-related conditions

Agent loop



Keep ReAct bounded. Start with a maximum of 3–5 evidence-retrieval/reasoning iterations per agent.

Tools

Feature	Status	Preferred	Alternative
Orchestration	MVP	LangGraph	Simple async Python state machine
LLM	MVP	Free/local model where adequate; otherwise low-cost API	OpenAI-compatible API provider
Structured output	MVP	Pydantic	JSON Schema
Embeddings	Advanced	all-MiniLM-L6-v2	BGE-small / other lightweight model

10. Evidence Arbitration & Hypothesis Ranking

Each domain agent returns a structured HypothesisFinding rather than an unrestricted narrative.

```
HypothesisFinding
├─ hypothesis
├─ supporting_evidence[]
├─ contradicting_evidence[]
├─ relevant_events[]
├─ missing_evidence[]
├─ assumptions[]
├─ reasoning_summary
├─ confidence
└─ uncertainty
```

MVP ranking dimensions

Dimension	Meaning
Evidence support	How strongly retrieved evidence supports the hypothesis
Temporal consistency	Whether the hypothesis fits the reconstructed sequence
Source reliability	Reliability assigned to the evidence source/type
Causal support	Whether graph relationships support the proposed contribution
Evidence completeness	Penalty for important missing evidence
Contradiction penalty	Penalty for strong conflicting observations

Important methodological rule

Agent confidence should not automatically become a Bayesian probability. In MVP, use a transparent weighted evidence score. Bayesian inference is an Advanced module with explicitly defined probability assumptions.

Advanced arbitration

Dempster-Shafer combination may be added when TRACE needs formal combination of uncertain/conflicting evidence. It should be implemented only after the basic scoring pipeline is validated.

11. Causal Graph & Probabilistic Reasoning

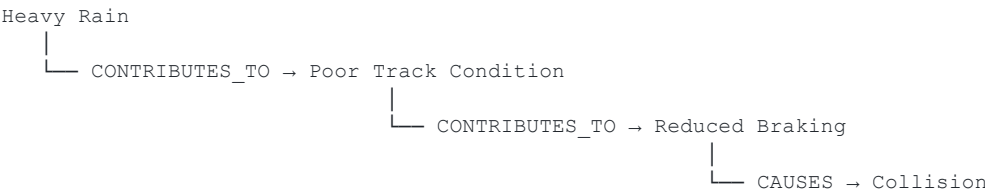
TRACE uses Neo4j for causal/event structure and, optionally, pgmpy for probabilistic inference. They serve different purposes.

Graph model

Node types:
EVENT
FACTOR
CONDITION
ACTION
SYSTEM_STATE
HYPOTHESIS
OUTCOME

Edge types:
PRECEDES
CAUSES
CONTRIBUTES_TO
ENABLES
CORRELATES_WITH
CONTRADICTS

Example



Graph features

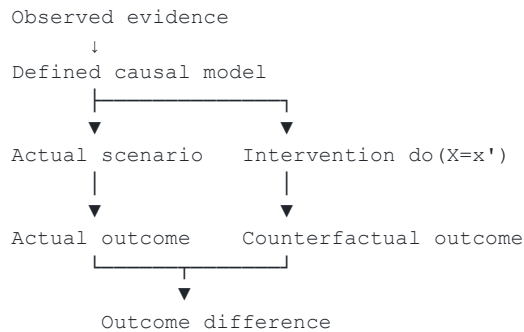
Feature	Status	Preferred	Alternative
Graph storage	MVP	Neo4j Community	NetworkX for very small prototype
Path tracing	MVP	Cypher variable-length paths	NetworkX shortest_path
Graph visualization	MVP	React Flow / Cytoscape.js	D3.js
Bayesian inference	Advanced	pgmpy	PyMC for more flexible models
Causal discovery	Future	PC/GES research implementation	DoWhy causal discovery tools

Causal edges must carry evidence references and confidence/strength metadata. A graph without evidence-linked edges is only a visualization, not a forensic reasoning structure.

12. Counterfactual Analysis

Counterfactual analysis is Advanced. It should not be implemented as simply deleting a node and rerunning a graph.

Correct conceptual workflow



Example

Actual: braking begins at time T. Counterfactual: braking begins 5 seconds earlier. TRACE evaluates how the modeled trajectory/outcome changes under that intervention.

Tools

Feature	Status	Preferred	Alternative
Causal intervention	Advanced	DoWhy	EconML / custom SCM
Probabilistic model	Advanced	pgmpy	PyMC
Scenario comparison	Advanced	Python + stored simulation state	custom domain simulator

Reporting language

TRACE should describe results as model-based counterfactual evidence. It should not claim that a counterfactual experiment proves real-world causation.

13. Conflicts, Evidence Gaps & Rule-Based Anomalies

These features are inexpensive and were strongly informed by the previous forensic project. They should be Core rather than postponed.

A. Evidence conflict detection

- Signal log says RED while another source reports GREEN at the same aligned time.
- Maintenance report says normal while sensor evidence indicates abnormal condition.
- Two sources assign incompatible states to the same entity/event.
- A reported event occurs at a time inconsistent with another high-confidence event.

B. Rule-based railway anomalies

Rule	Example
Impossible sequence	Brake release recorded before brake application
Speed threshold	Train speed exceeds permitted threshold
Temporal prerequisite	Collision recorded before train entered section
Maintenance interval	Inspection overdue beyond configured threshold
Signal transition	Invalid or unexpected signal-state transition
Physical plausibility	Sensor value outside configured physical range
Clock inconsistency	Source timestamp moves backward unexpectedly

Implementation

Start with plain Python rules and configuration files. A rule should return rule_id, severity, affected events, evidence references and human-readable explanation.

C. Evidence gaps / Known Unknowns

KNOWN UNKNOWN

Missing:

Brake diagnostic record

Impact:

High

Affected hypotheses:

Delayed braking

Brake-system failure

Recommended next evidence:

Brake-system diagnostic / maintenance record

14. Investigator UI, Explainability & Reporting

TRACE should be a case workspace rather than a collection of disconnected research visualizations.

Recommended navigation

```

Dashboard
Cases
└─ Case
    │ ─ Overview
    │ ─ Evidence
    │ ─ Timeline
    │ ─ Hypotheses
    │ ─ Causal Graph
    │ ─ Evidence Gaps
    │ ─ Findings
    │ ─ Report
    └─ Audit
  
```

Key screens

Screen	Purpose
Case Overview	Status, incident metadata, evidence count, top findings, conflicts and gaps
Evidence Explorer	Search/filter evidence and inspect source/provenance
Timeline	Unified event sequence with source lanes, confidence and conflict markers
Hypothesis Comparison	Side-by-side supporting/contradicting/missing evidence
Causal Graph	Explore cause-effect paths and evidence-linked edges
Evidence Gaps	Known unknowns and potentially high-value missing evidence
Findings	Accepted/rejected/review-needed conclusions
Report	Generate investigation summary and evidence appendix
Audit	Show important case and system actions

Layered explanations

Every important result should support: Quick summary → Investigator explanation → Evidence details → Technical reasoning.

Frontend tools

Feature	Status	Preferred	Alternative
Frontend	MVP	React + TypeScript	Next.js
Timeline	MVP	Plotly React	D3.js
Graph	MVP	Cytoscape.js / React Flow	D3.js
PDF	MVP	ReportLab	WeasyPrint
Live progress	MVP/optional	Polling first	WebSocket / SSE

15. Data, Storage & API Architecture

Core relational entities

Table	Purpose
cases	Investigation case metadata/status
evidence_artifacts	Original evidence metadata, hash and provenance
evidence_records	Parsed source observations
events	Normalized investigation events
hypotheses	Hypothesis definitions/status
agent_findings	Structured domain-agent results
findings	Final/meta-level investigation findings
evidence_links	Finding/event/hypothesis-to-evidence relationships
anomalies	Rule-based or statistical anomalies
audit_log	State-changing actions and model versions

API groups

Endpoint group	Examples
Cases	POST /cases, GET /cases/{id}
Evidence	POST /cases/{id}/evidence, GET /evidence/{id}
Timeline	GET /cases/{id}/timeline
Investigation	POST /cases/{id}/investigate, GET /cases/{id}/findings
Hypotheses	GET /cases/{id}/hypotheses, GET /hypotheses/{id}
Graph	GET /cases/{id}/graph, GET /graph/path
Reports	POST /cases/{id}/reports, GET /reports/{id}
Audit	GET /cases/{id}/audit

Preferred stack

- FastAPI + Pydantic + SQLAlchemy
- PostgreSQL for shared deployment; SQLite for very small local development
- Neo4j Community for graph storage
- Local filesystem initially; MinIO/S3-compatible storage later
- Docker Compose for reproducible development

16. Technology Stack & Alternatives

Selection rule: prefer free, mature and low-complexity tools. Use an alternative only when the preferred choice creates a real limitation.

Layer	Preferred	Alternative	Reason
Backend	FastAPI	Flask	Simple async API + OpenAPI
Validation	Pydantic	dataclasses	Typed contracts
Database	PostgreSQL	SQLite	Scalable relational core
ORM	SQLAlchemy	SQLModel	Mature ecosystem
Graph	Neo4j Community	NetworkX	Purpose-built graph queries
Parsing	pandas/PyMuPDF	Polars/pdfplumber	Free + mature
Temporal	dateutil/zoneinfo/SciPy	Pendulum/statsmodels	Simple statistical tools
Agents	LangGraph	custom async state machine	Explicit graph/state orchestration
MCP	Python MCP SDK	FastAPI adapter	Native context/tool interface
LLM	Local/free model where adequate	Low-cost API	Control cost and privacy
Embeddings	all-MiniLM-L6-v2	BGE-small	Lightweight CPU-friendly
Graph UI	Cytoscape.js	React Flow/D3	Interactive graph without 3D complexity
Charts	Plotly	D3.js	Fast implementation
PDF	ReportLab	WeasyPrint	Direct programmatic PDF
Deployment	Docker Compose	local Python/Node	Reproducibility
Testing	pytest	unittest	Simple Python testing

Cost policy

No core feature should require a paid API. Paid LLM/API services may be used during development only if needed and should have a local/free fallback documented in configuration.

17. Phase-wise Implementation Plan

Phase 0 — Architecture Freeze

- Finalize schemas: Case, EvidenceArtifact, EvidenceRecord, Event, HypothesisFinding, Finding.
- Finalize API contracts and graph model.
- Create synthetic railway evidence specification.
- Define evaluation metrics and ground truth.

Phase 1 — Foundation & Evidence

- FastAPI + database setup.
- Case creation and evidence upload.
- SHA-256 + provenance + audit log.
- CSV/JSON/TXT/PDF parsing.
- Evidence Explorer and Evidence Details UI.

Phase 2 — Events & Temporal Engine

- Normalization schemas.
- Event extraction.
- Timezone/timestamp normalization.
- Clock offset and drift correction.
- Unified timeline.
- Initial rule-based anomalies.

Phase 3 — Investigation Engine

- MCP evidence tools.
- LangGraph state.
- Four domain agents.
- Bounded evidence retrieval loops.
- Structured HypothesisFinding.
- Meta-agent and evidence-weighted ranking.

Phase 4 — Causal Investigation

- Neo4j graph construction.
- Evidence-linked causal edges.
- Causal path tracing.
- Hypothesis comparison.
- Evidence conflicts and gaps.
- Layered explanation.

Phase 5 — UI, Reports & Evaluation

- Integrated investigator dashboard.
- Finding review/accept/reject.
- PDF investigation report.
- End-to-end synthetic accident scenarios.
- Module-level evaluation.

Phase 6 — Advanced Extensions

- DTW.
- Bayesian Network.
- Dempster-Shafer.
- Counterfactual interventions.

- Semantic retrieval.
- Similar-case retrieval.

18. Testing, Evaluation, Scalability & Risks

Evaluation should be modular rather than relying on one overall 'accuracy' number.

Module	Metric
Evidence parsing	Parsing success rate
Event extraction	Precision / Recall / F1
Clock correction	Mean absolute timestamp error
Event ordering	Ordering accuracy
Rule anomalies	Precision / Recall / false-alarm rate
Conflict detection	Precision / Recall
Hypothesis ranking	Top-1 / Top-3 accuracy
Graph	Relationship accuracy / query latency
Traceability	% findings with valid evidence paths
System	End-to-end case processing time

Synthetic dataset strategy

- Generate controlled accident cases with known causal chains.
- Create multiple source clocks with intentional offsets/drift.
- Inject missing records and contradictory observations.
- Vary evidence quality and source reliability.
- Keep ground-truth event order and causal factors hidden from the system during testing.

Scalability path

MVP:

Modular monolith + PostgreSQL + Neo4j

↓ if processing grows

Background job queue + worker

↓ if services become bottlenecks

Separate evidence / temporal / investigation services

↓ at real deployment scale

Object storage + event bus + distributed workers

Risks and mitigation

Risk	Mitigation
LLM hallucination	Evidence-grounded retrieval + structured output + human review
False causality	Separate correlation from causal claims; use explicit causal model
Bad timestamps	Store raw + corrected timestamps and temporal confidence
Missing evidence	Known-unknown panel + completeness penalty
Conflicting evidence	Explicit contradiction objects; never silently merge
Overengineering	MVP-first feature freeze
High API cost	Local/free model fallback; cache structured results
Graph complexity	Small typed schema; add algorithms only when justified
Data privacy	Local deployment and controlled evidence access

19. Final Feature Freeze / Build Checklist

MVP feature freeze

Feature	Build
Case workspace	✓
Evidence upload + explorer	✓
SHA-256 + custody + audit	✓
Evidence → Record → Event model	✓
CSV/JSON/TXT/PDF support	✓
Timestamp normalization	✓
Clock offset/drift correction	✓
Unified timeline	✓
Rule-based anomalies	✓
Evidence conflict detection	✓
MCP evidence tools	✓
Train/Driver agent	✓
Signalling agent	✓
Track/Infrastructure agent	✓
Environment agent	✓
Meta-agent	✓
Evidence-weighted hypothesis ranking	✓
Evidence gaps / known unknowns	✓
Neo4j causal/event graph	✓
Causal path tracing	✓
Hypothesis comparison	✓
Layered explanations	✓
Investigator accept/reject/review	✓
PDF report	✓

Advanced feature gate

Do not begin advanced work until the complete MVP workflow succeeds on at least one fully synthetic end-to-end accident case and all major findings can be traced back to source evidence.

Advanced order of implementation

1. DTW → 2. Bayesian Network → 3. Dempster-Shafer → 4. semantic retrieval → 5. counterfactuals → 6. similar-case retrieval → 7. dynamic agents.

Final TRACE identity

TRACE is not an LLM that writes an accident story. It is a structured investigation pipeline in which deterministic evidence processing, temporal reconstruction, domain-specific agents, graph reasoning and human review work together to produce evidence-grounded hypotheses.

Reference basis: This blueprint incorporates lessons from the previously reviewed ADCSIS project, particularly its case-centric workflow, evidence integrity, graph-path exploration, hypothesis comparison, explainability, auditability and human-in-the-loop approach. The previous project also demonstrated the importance of explicitly documenting implementation limits rather than treating every proposed architecture component as implemented.